

Rebuilding EventScore

Standing the whole thing up from the repository, on new infrastructure, from nothing.

- Everything needed is in the repository
- About a day, most of it waiting for DNS
- Read the technical reference first

Read the technical reference before changing anything. Several decisions in the scoring and progression code look like preferences and are load-bearing. Reversing them produces results that are silently wrong rather than obviously broken, and that document lists which ones and why.

What you have been given

In the repository	What it is
src/	The application. Next.js 16, App Router, TypeScript.
db/01-schema.sql	The complete database structure: tables, row-level security policies, functions and triggers.
backups/	Nightly database dumps, thirty days. Schema and data as separate files.
.github/workflows/	Maintenance, backup and schema export jobs.
docs/	These documents.
.env.example	The environment variables you need, without values.
README.md	The short version of this document.

What you have not been given

Missing	What to do
Environment variable values	Create your own projects, or ask the owner for the four values.

Missing	What to do
Contestant photographs	They live in Supabase storage, not in any backup. If they matter, ask for a copy of the event-media bucket.
Account access	Supabase, Vercel and the domain registrar are separate logins.

Standing it up

1. The database

- 1 Create a Supabase project. **Choose an EU region:** the privacy policy promises EEA storage and the region cannot be changed later.
- 2 Save the database password somewhere permanent. It cannot be recovered.
- 3 Open the SQL Editor and run the contents of **db/01-schema.sql**. That creates every table, policy, function and trigger.
- 4 Create a storage bucket called **event-media** and make it **private**.
- 5 Add the storage policy below.

```
create policy "own media" on storage.objects
  for all to authenticated
  using (bucket_id = 'event-media' and owner = auth.uid())
  with check (bucket_id = 'event-media' and owner = auth.uid());
```

The storage bucket is not in the schema dump. It is a Supabase storage object rather than a table, so it has to be created by hand. Miss it and everything photograph-related fails quietly.

2. The application

```
git clone the repository
cd eventscore
npm install
cp .env.example .env.local
```

Fill in `.env.local` from Supabase, Project Settings, API. `CRON_SECRET` can be any long random string. Then:

```
npm run dev
```

3. The first owner account

- 1 In Supabase, Authentication, Users, add a user. Tick **Auto Confirm User**.
- 2 A trigger creates the matching profiles row automatically.
- 3 In the SQL Editor, promote them.

```
update profiles set role = 'owner' where email = 'you@example.com';
```

Sign in at /login. Backstage appears in the header. From then on every account is created from within the app.

4. Deploying

- 1 Connect the repository to Vercel, Netlify or anything supporting Next.js server actions.
- 2 Add all four environment variables in the host's settings. They are not in git.
- 3 **Set the function region to match the database.** Dublin for eu-west-1. Getting this wrong makes every query cross an ocean twice and the app feels broken.
- 4 Deploy, then check judge sign-in specifically, because it is the one path needing the service role key.
- 5 Point DNS at the host. A CNAME for a subdomain.

Vercel's Hobby plan is non-commercial only. If this is being run commercially it needs Netlify Free, which permits it, or Vercel Pro.

5. Scheduled work

Add these as repository secrets under Settings, Secrets and variables, Actions:

Secret	Value
SUPABASE_URL	Same as NEXT_PUBLIC_SUPABASE_URL
SUPABASE_ANON_KEY	Same as NEXT_PUBLIC_SUPABASE_ANON_KEY
CRON_SECRET	Same as the app
APP_URL	The live URL, no trailing slash
SUPABASE_DB_URL	The session pooler connection string. The direct one is IPv6 only and GitHub runners cannot reach it.

Then run each workflow manually once from the Actions tab to confirm it works, rather than discovering it at three in the morning.

Restoring data

Only if you want the previous data. A fresh build does not need it.

```
gunzip backups/schema-DATE.sql.gz backups/data-DATE.sql.gz
psql "POOLER_URL" -f backups/schema-DATE.sql
psql "POOLER_URL" -f backups/data-DATE.sql
```

Schema first, then data. The dumps include drop statements, so this overwrites whatever is there. Photographs are separate and need uploading to the storage bucket by hand.

Verifying it works

Walk the whole thing rather than trusting a clean build. Most of the serious bugs in this project were invisible until somebody used it properly.

- Sign in as the owner. Backstage loads.
- Create a customer account from backstage. The temporary password appears.
- Sign in as that customer. The terms gate appears and accepts.
- Create an event. It gets a six-character code automatically.
- Add two rounds, the second marked as ending the event.
- Add categories to both rounds.
- Add four contestants, at least one with a photograph.
- Add one judge. A link, QR code and PIN appear.
- Open the judge link in a private window. The judge's name and photo show.
- Enter the PIN. The scorecard opens with all four contestants.
- Score everyone, tap Review, tap Submit round.
- The shortlist appears with the right number of contestants.
- Score the final round and submit. Rankings appear with a podium.
- Export the PDF. Every round and mark is in it.
- Turn wifi off mid-scoring. The offline banner appears and marks queue.
- Turn wifi on. The badge goes green within a second or two.

Test the photograph path specifically. It touches storage, signed URLs and the bucket policy, and it fails silently if the bucket or its policy is missing. A contestant with no photograph looks identical to a broken one.

Moving off Supabase

Harder than moving hosts, because Supabase provides four things. The database itself is standard Postgres and moves cleanly.

Piece	Effort	What it takes
Database	Low	pg_dump and restore to any Postgres host. Policies, functions and triggers come with it.
Auth	High	Rewrite src/lib/supabase, plus the login, security and backstage actions. The largest single job.
Storage	Medium	Replace src/lib/media.ts. Preserve the signed-URL pattern; do not make the bucket public.
Judge sessions	None	Already custom. They only need a Postgres connection.

Where to be careful

Area	Why
src/lib/scoring.ts	A missing score row means zero, not absent. Changing that silently stops contestants advancing.
src/lib/tiebreak.ts	Ties are only detected at places that change the outcome. Widening it would interrupt events for ties nobody cares about.
The absence of an owner read policy	There is deliberately no policy letting an owner read event data. Adding one would break a promise made in the privacy policy.
Any useEffect with an interval	Never put router in a dependency array. It caused a request storm that made the app unusable.
Server actions	Do not call revalidatePath in an action the client also refreshes. The two race and the UI freezes.
ensureRoster	It refuses to rebuild a line-up once scoring has started. That guard prevents judges seeing different contestants.

If you are changing the schema

Run the change in the Supabase SQL Editor, then re-export so the repository stays true:

Actions tab, Export schema, Run workflow

That commits a fresh db/01-schema.sql. Without it the repository drifts from reality and the next person rebuilds the wrong thing.

Structural and data changes must be separate queries. Postgres plans a whole batch before executing, so an ALTER TABLE followed by an UPDATE using the new column fails on a column that does not exist yet.